

CIT 5940 Module 1 Programming Assignment

Optional Challenges

Assessment

This Extra Credit assignment is composed of two parts. There is one graded optional challenge and one ungraded challenge you can do if you want to dive deeper.

Graded Challenge: Implement circular dynamic arrays.

Ungraded Challenge: Minimize use of `@SuppressWarnings`. This would help you come up with a cleaner implementation.

Setup

For the graded challenge, you will download `CircularArrayList.java` from the Google Drive.

Submission

For this semester, unlimited submissions of this EC is allowed. Regardless, please make sure to test it thoroughly yourself before submitting.

Whether you try the graded challenge or not, you should upload `CircularArrayList.java` to Gradescope.

For the rest of the submission instructions, follow sections 2.1, 3.1 and 3.2 from the write-up for the regular assignment.

Optional Challenges

Graded Challenge: Implement Circular Dynamic Arrays

In this assignment, you have implemented a dynamic array, also known as an array list or a vector. This data structure is very good for use cases where you often add to the end of the list, but rarely to the beginning or middle.

It is also possible to write a “circular” dynamic array if we consider the first element as next of the last element. This would be useful if you wanted a list that is reasonably fast both at adding to the front of the list and at adding to the back of the list. (Thus, it would be a candidate implementation of a double-ended queue, an abstract datatype you will learn about in a later module.)

For a graphical visualization and hint at how to do this, see page 242 of the textbook.

Note that this would require a re-write of much of the code, but you still want your array list to expose the same basic interface as given in the starter code for this assignment.

In addition to the variables/functions in the regular assignment, you will have a `startIndex` variable that represents which physical index of the underlying `data` array is considered the logical start index for the circular array.

The following examples demonstrate the need for the `startIndex` variable and the expected behavior of some of the common operations. Below, the indices are those of the circular array, where index 0 represents the `startIndex`. The indices of the underlying array will always have 0 at the leftmost position initially. Notice how the front of the circular array, marked as index 0, is not necessarily at the front of the underlying array. If you don't keep track of the `startIndex`, you'll have no way of knowing where the beginning of the circular array is.

```
1      Original Array:  [ A,  B,  X,  C,  D,  null,  null,  null ]
2      Original Index:   0   1   2   3   4   5   6   7
3
4      addFront("Z");
5      New Array       :  [ A,  B,  X,  C,  D,  null,  null,  Z ]
6      Index           :   1   2   3   4   5   6   7   0
7
8      removeFront();
9      New Array       :  [ A,  B,  X,  C,  D,  null,  null,  null ]
10     Index           :   0   1   2   3   4   5   6   7
11
```

```

12  removeFront();
13  New Array      :  [ null, B, X, C, D, null, null, null ]
14  Index         :      7      0  1  2  3  4      5      6
15
16  add("Y");
17  New Array      :  [ null, B, X, C, D, Y, null, null ]
18  Index         :      7      0  1  2  3  4      5      6
19
20  remove("C");
21  New Array      :  [ null, B, X, D, Y, null, null, null ]
22  Index         :      7      0  1  2  3      4      5      6
23
24  removeFront();
25  New Array      :  [ null, null, X, D, Y, null, null, null ]
26  Index         :      6      7      0  1  2      3      4      5
27
28  removeFront();
29  New Array      :  [ D, Y, null, null ]
30  Index         :      0  1      2      3
31

```

In the last example, we needed to shrink the underlying array. Whenever we shrink or increase the capacity of the array, we position the beginning element of the circular array at index 0 in the underlying array, ie. `startIndex == 0`.

You will also have 3 more functions that you need to implement, `addFront`, `removeFront` and `removeBack`. All specifications are the same as the regular assignment, unless they are specified here.

- Whenever `increaseCapacity` and `shrinkCapacity` are called, they set `startIndex` to 0. i.e. the 0th index of the new array contains the first element of the old circular array and so on.
- If the array is empty and `removeFront` or `removeBack` are called, they should throw `NoSuchElementException`.
- `addFront` should first check if there is enough capacity to add an element, and once there is, it should decrement `startIndex` and add the element appropriately.
- `removeFront` should update the `startIndex` to remove the front element.
- All of the add and remove functions should increase/shrink capacity as required.
- There is no minimum capacity for the underlying array, i.e., a capacity of 0 is valid. (This is the same as M1PA.) Thus, when calculating the new size of the array for `increaseCapacity`, use the same computation as is used in M1PA:

`newCapacity = max(2 * data.length, INITIAL_CAPACITY)`

- The `get` function should return the element corresponding to the circular list, and not the underlying data array.
- The circular array list should also be generic.

Note that we will have performance tests in addition to functional tests, so your functions need to be efficient. In essence, the functions to add and remove from the front/back shouldn't require any shifting of elements. The `increaseCapacity` and `shrinkCapacity` functions won't be called that often, so if you use an implementation similar to what was used for the regular assignment, that should be ok.

Ungraded Challenge: Minimize use of `@SuppressWarnings`

Take a look at your code. In how many places are you using `@SuppressWarnings("unchecked")` to suppress the unsafe cast warning? It turns out, you can reduce this down to just one single place. Try to figure out how to do this, and implement it.